Storage and files*

Distributed Data Processing Environments

Lab Guide 4

This session aims at converting, storing, and retrieving data files in the cloud. It consumes education credits: this is the one session that uses an e2-medium instance, as the data no longer fits comfortably in a smaller one.

Steps First describe an instance and a bucket together:

1. Copy the folder you wrote in Lab 3 into a new one.
2. Change the machine type to e2-medium.
3. Add a `google_storage_bucket`,[1] in location *US*. Bucket names are unique across all of Google Cloud, so include your project in the name.
4. Add `force_destroy` to the bucket, so that `tofu destroy` can remove it while it still holds objects. Explain why that is not the default.
5. Add a `google_compute_disk` of 10 GB,[2] and attach it to the instance.
6. Add a startup script to the instance that installs DuckDB[3] where every user can run it. The installer puts it in the home directory of whoever runs it, which is not what you want here.
7. Run `tofu apply`, connect with `gcloud compute ssh`, and confirm that `duckdb` runs.
8. Find the new disk with `lsblk`, and again under `/dev/disk/by-id/`. Explain which of the two names you should use. Give it a file system, mount it, and make it writable by your user.

Now read data that is kept in the bucket:

1. From your own machine, upload the Bundesliga dataset of Lab 0 with `gcloud storage cp`.
2. In DuckDB on the instance, prepare access to the bucket:

```
INSTALL gcs FROM community;
LOAD gcs;
CREATE SECRET (TYPE gcp, PROVIDER credential_chain);
```

3. Count the rows of the file, reading it directly from the bucket:
   `SELECT count(*) FROM 'gcss://`*bucket*`/matches.csv';`
4. Notice that no key and no password was given and that the same statements fail on your own machine.
5. Compute the goals scored by one team in one season. Then check that figure by counting the rows yourself, without the query.
6. Reconcile the two numbers. If they differ, check the exact spelling of the team name, how the lines of the file end, and whether a match appears in the file once or twice.

*Author mode in the first part. Predict before measuring in the last. See AI protocol.

[1] `https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/storage_bucket`

[2] `https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_disk`

[3] `https://duckdb.org/install/`

Now obtain the same data in different formats and places:

1. Generate a larger dataset inside DuckDB:

   ```
   INSTALL tpch; LOAD tpch;
   CALL dbgen(sf = 1);
   ```

2. Export the *lineitem* table to the disk of the instance, as CSV[4] and as Parquet.[5]

3. Compare the sizes of the two files, and explain the difference.

4. Copy both files to the bucket.

Finally, measure what the choice costs you:

1. Write one query that aggregates over the whole table.

2. Predict which of the four combinations of place and format will be fastest, and which slowest. Write your order down before measuring.

3. Ask the agent for its own prediction, and keep it next to yours.

4. Measure each of the four with `.timer on`. It is a command of the shell and not of SQL, so it only works when you type it, or feed it on standard input.

5. Explain the result. Which mattered more, where the file was, or how it was written?

Now compare the two kinds of storage you have been using:

1. You chose a size for the disk, but never for the bucket. Find what each of them costs you, and explain which one you pay for when it is empty.

2. Append one row to the CSV file on the disk. Then append one row to the same file in the bucket. Explain why one of these is a small operation and the other is not.

3. Explain how many instances could read the file on the disk at once, and how many could read the one in the bucket.

4. Delete the instance, but not the disk, from the console. Explain what remains, and what it still costs.

At the end of the session, run `tofu destroy` and follow the teardown checklist in the cloud protocol.

Learning Outcomes Deploy data in instance disks and cloud object storage. Distinguish block from object storage by how each is addressed and paid for. Relate storage location and file formats with processing performance. Grant access to data without persistent credentials.

[4] `https://duckdb.org/docs/stable/guides/file_formats/csv_export`
[5] `https://duckdb.org/docs/stable/guides/file_formats/parquet_export`